LIBXS

LIBXS is a portable C library providing building blocks for memory operations, numerics, synchronization, and more -- with a focus on performance and minimal dependencies. Targets x86-64, AArch64, and RISC-V; requires only a C89 compiler. Originally developed as part of LIBXSMM.

Functionality

| Domain | Header | Description | |||| | Memory | `libxs_mem.h` | Byte comparison, matrix copy/transpose, alignment queries | | Permutation | `libxs_perm.h` | Co-prime shuffling, smooth row permutations | | Hashing | `libxs_hash.h` | CRC32-based hashing, Adler-32, string hashing | | String | `libxs_str.h` | Edit distance, substring search, word similarity, formatting | | GEMM | `libxs_gemm.h` | Batched dense GEMM (strided, pointer-array, grouped) | | Math | `libxs_math.h` | Matrix comparison, GCD/LCM, coprime, BF16 conversion | | MHD | `libxs_mhd.h` | Read/write MetaImage (MHD/MHA) files | | Histogram | `libxs_hist.h` | Thread-safe histogram with running statistics | | Malloc | `libxs_malloc.h` | Pool-based allocator (steady-state, no system calls) | | RNG | `libxs_rng.h` | Thread-safe pseudo-random number generation (SplitMix64) | | Sync | `libxs_sync.h` | Portable atomics, locks, TLS, and file locking | | Timer | `libxs_timer.h` | High-resolution timing via calibrated TSC | | CPUID | `libxs_cpuid.h` | CPU feature detection (SSE to AVX-512, AArch64, RISC-V) | | Registry | `libxs_reg.h` | Thread-safe key-value store with per-thread caching | | Utils | `libxs_utils.h` | ISA feature gates, bit-scan, SIMD helpers |

See also: Fortran Interface, Scripts.

Build

```
make GNU=1
```

The library is compiled for SSE4.2 by default but dynamically dispatches to the best ISA available at runtime (up to AVX-512). Use `SSE=0` to compile natively for the build host.

| Variable | Default | Description | |||| | GNU | 0 | Use GNU GCC-compatible compiler | | DBG | 0 | Debug build | | SYM | 0 | Include debug symbols (-g) | | SSE | 1 | x86 baseline: 0=native, 1=SSE4.2 (portable) |

CMake is also supported (header-only or library):

```
cmake -S . -B build -DLIBXS_HEADER_ONLY=ON
cmake --build build
```

Installation

Install into a chosen prefix:

```
make GNU=1 -j $(nproc) install PREFIX=$HOME/libxs
```

This installs headers, the Fortran module, the static and shared libraries, and the header-only source tree under `PREFIX`.

Out-of-tree builds are also supported:

```
mkdir /tmp/libxs-build && cd /tmp/libxs-build
make -j $(nproc) -f /path/to/libxs/Makefile
```

Usage

Library -- link against `libxs.a` (or `.so`) and include the desired headers:

```
#include <libxs_mem.h>
#include <libxs_timer.h>
```

Header-only (explicit) -- include `libxs_source.h` (no separate library needed). Safe to include from multiple translation units:

```
#include <libxs_source.h>
```

Header-only (implicit) -- compile with `-DLIBXS_SOURCE` and any LIBXS public header automatically includes the implementation. No special include order is required. When used through LIBXSTREAM without a pre-built library (`-DLIBXSTREAM_SOURCE`), `LIBXS_SOURCE` is implied automatically.

Fortran -- use the provided module (documentation):

```
USE :: libxs, ONLY: libxs_memcmp
```

Samples

| Sample | Description | ||| | gemm | Batched DGEMM (strided, pointer-array, grouped) with OMP | | memory | Benchmarks for comparison, matrix copy, and transpose | | ozaki | Ozaki-scheme low-precision GEMM with intercepted BLAS | | registry | Registry dispatch microbenchmark | | scratch | Pool allocator vs system malloc | | shuffle | Shuffling strategies comparison | | sync | Lock implementation microbenchmarks |

License

BSD 3-Clause

LIBXS Domains

CPU Identification

Header: `libxs_cpuid.h`

Portable CPU feature detection for x86-64, AArch64, and RISC-V targets. Returns an ISA level that can be compared numerically -- higher values indicate more capable instruction sets. x86 levels use thermometer ordering: higher numeric value implies all features of lower levels. AVX10/256 sits below AVX512 because it lacks 512-bit vectors.

ISA Constants

| Constant | Value | Architecture | |||| | `LIBXS_TARGET_ARCH_UNKNOWN` | 0 | Unknown / unsupported | | `LIBXS_TARGET_ARCH_GENERIC` | 1 | Portable scalar baseline | | `LIBXS_X86_GENERIC` | 1002 | x86-64 baseline | | `LIBXS_X86_SSE3` | 1003 | SSE3 | | `LIBXS_X86_SSE42` | 1004 | SSE4.2 | | `LIBXS_X86_AVX` | 1005 | AVX | | `LIBXS_X86_AVX2` | 1006 | AVX2 + FMA | | `LIBXS_X86_AVX10_256` | 1030 | AVX10.1/256: all features, 256-bit max (Sierra Forest) | | `LIBXS_X86_AVX512` | 1100 | AVX-512 (F + CD + DQ + BW + VL + VNNI) | | `LIBXS_X86_AVX512_AMX` | 1105 | AVX-512 + AMX (TILE + INT8 + BF16); Sapphire/Granite Rapids | | `LIBXS_X86_AVX512_INT8` | 1110 | AVX-512 + AVX-VNNI-INT8 (VPDPBUUD/BSSD) | | `LIBXS_X86_AVX10_512` | 1200 | AVX10.1/512 + AMX + INT8: full feature set | | `LIBXS_AARCH64` | 2001 | ARMv8.1 baseline | | `LIBXS_AARCH64_SVE128` | 2201 | SVE 128-bit | | `LIBXS_AARCH64_SVE256` | 2301 | SVE 256-bit | | `LIBXS_AARCH64_SVE512` | 2401 | SVE 512-bit | | `LIBXS_RV64_MVL128` | 3001 | RISC-V RVV 128-bit | | `LIBXS_RV64_MVL256` | 3002 | RISC-V RVV 256-bit | | `LIBXS_RV64_MVL128_LMUL` | 3003 | RISC-V RVV 128-bit (non-unit LMUL) | | `LIBXS_RV64_MVL256_LMUL` | 3004 | RISC-V RVV 256-bit (non-unit LMUL) | | `LIBXS_X86_ALLFEAT` | 1999 | x86 sentinel (all features) | | `LIBXS_AARCH64_ALLFEAT` | 2999 | AArch64 sentinel (all features) | | `LIBXS_RV64_ALLFEAT` | 3999 | RISC-V sentinel (all features) |

Types

```
typedef struct libxs_cpuid_t {
  char model[256];   /* CPU model name from OS */
  int constant_tsc;  /* non-zero if TSC is invariant */
} libxs_cpuid_t;
```

Functions

```
int libxs_cpuid(libxs_cpuid_t* info);
```

Detect the ISA level of the current platform. Optionally fills `info` with the CPU model name and TSC capability. Returns a constant from the table above.

```
const char* libxs_cpuid_name(int id);
```

Returns a human-readable string for the given ISA constant, e.g., `"avx2"`.

```
int libxs_cpuid_id(const char* name);
```

Translates a name (e.g., `"avx512"`) to the corresponding ISA constant.

```
int libxs_cpuid_vlen(int id);
```

Returns the SIMD vector length in bytes for the given ISA level (0 for scalar-only targets).

```
int libxs_cpuid_amx_enable(void);
```

Request AMX tile state (XTILE_DATA) from the OS. Must be called before any AMX tile instruction. Returns `EXIT_SUCCESS` if tiles are ready (or were already enabled), `EXIT_FAILURE` on unsupported platform or OS refusal. Not called automatically by `libxs_cpuid` to avoid the per-thread XSAVE overhead for non-AMX workloads.

project: LIBXS author: Intel Corporation license: BSD 3-Clause License summary: Small utility library. project_github: https://github.com/hfp/libxs/ project_download: https://github.com/hfp/libxs/releases/latest/ favicon: ../.theme/img/favicon.png css: ../.theme/ford.css output_dir: fortran src_dir: ../include search: true

Reference Documentation

GEMM: Matrix Multiplication

Header: `libxs_gemm.h`

Batched general matrix-matrix multiplication (GEMM). Operations are expressed as C := alpha * op(A) * op(B) + beta * C, where op() is an optional transpose. The caller can supply MKL JIT, LIBXSMM, or BLAS kernels; otherwise a built-in default kernel (auto-vectorized) is used.

Types

```
typedef void (*libxs_gemm_dblas_t)(
  const char* transa, const char* transb,
  const int* m, const int* n, const int* k,
  const double* alpha, const double* a, const int* lda,
                       const double* b, const int* ldb,
  const double* beta,        double* c, const int* ldc);
```

Standard Fortran BLAS dgemm signature (e.g., `dgemm_`).

```
typedef void (*libxs_gemm_sblas_t)(
  const char* transa, const char* transb,
  const int* m, const int* n, const int* k,
  const float* alpha, const float* a, const int* lda,
                      const float* b, const int* ldb,
  const float* beta,        float* c, const int* ldc);
```

Standard Fortran BLAS sgemm signature (e.g., `sgemm_`).

```
typedef void (*libxs_gemm_djit_t)(void* jitter,
  const double* a, const double* b, double* c);
typedef void (*libxs_gemm_sjit_t)(void* jitter,
  const float* a, const float* b, float* c);
```

MKL JIT kernel signatures. Shape, alpha, beta, and transpose info are baked into the jitter handle.

```
typedef struct libxs_gemm_param_t {
  void* op[4]; const void* a[6]; const void* b[6]; void* c[6];
} libxs_gemm_param_t;
```

XGEMM parameter struct, layout-compatible with `libxsmm_gemm_param`. Only `a[0]` (a.primary), `b[0]` (b.primary), and `c[0]` (c.primary) are used for plain GEMM.

```
typedef void (*libxs_gemm_xfn_t)(const void*);
```

Opaque XGEMM kernel signature: `void(const libxs_gemm_param_t*)`.

```
typedef enum libxs_gemm_flags_t {
  LIBXS_GEMM_FLAGS_DEFAULT = 0,
  LIBXS_GEMM_FLAG_NOLOCK = 1
} libxs_gemm_flags_t;
```

Flags controlling batch synchronization. By default, `_task` variants synchronize C-matrix updates. Set `LIBXS_GEMM_FLAG_NOLOCK` when no duplicate C pointers exist.

```
typedef struct libxs_gemm_shape_t {
  libxs_data_t datatype;
  int transa, transb;
  int m, n, k, lda, ldb, ldc;
  double alpha, beta;
} libxs_gemm_shape_t;
```

GEMM shape: problem geometry, transpose flags, and scalar coefficients. Alpha and beta are stored as double regardless of datatype (float values are promoted without loss). Also serves as registry key when caching dispatched configurations (all-int fields avoid padding).

```
typedef struct libxs_gemm_config_t {
  libxs_gemm_dblas_t dgemm_blas;
  libxs_gemm_sblas_t sgemm_blas;
  libxs_gemm_djit_t dgemm_jit;
  libxs_gemm_sjit_t sgemm_jit;
  libxs_gemm_xfn_t xgemm;
  void* jitter;
  libxs_gemm_flags_t flags;
  libxs_gemm_shape_t shape;
} libxs_gemm_config_t;
```

Configuration supplying GEMM kernels. Pass NULL to batch functions to use the built-in default kernel (auto-vectorized, no BLAS dependency). Kernel selection priority:

1. JIT kernel (`dgemm_jit`/`sgemm_jit` + `jitter`) if non-NULL,
2. XGEMM kernel (`xgemm`) if non-NULL,
3. BLAS kernel (`dgemm_blas`/`sgemm_blas`) if non-NULL,
4. built-in default kernel.

Only the function pointers matching the datatype need to be set. The `shape` member is populated by `libxs_gemm_dispatch`.

Functions

```
int libxs_gemm_dispatch(
  libxs_gemm_config_t* config,
  libxs_data_t datatype, char transa, char transb,
  int m, int n, int k, int lda, int ldb, int ldc,
  const void* alpha, const void* beta,
  void* registry /* = NULL */);
```

Dispatch a GEMM kernel and populate `config` accordingly. With MKL JIT (highest priority): sets `config->dgemm_jit` or `sgemm_jit` + `jitter`. With LIBXSMM (fallback): sets `config->xgemm` via `libxsmm_dispatch_gemm`. Without either: config is unchanged (falls through to BLAS/default). The caller should memset config to zero before the first call. Returns `EXIT_SUCCESS` on success, `EXIT_FAILURE` when no JIT backend is available. If `registry` is non-NULL, dispatched configs are cached by shape: a hit copies the cached config, a miss dispatches and stores.

```
int libxs_gemm_ready(const libxs_gemm_config_t* config);
```

Check whether a dispatched config holds a usable kernel. Returns nonzero if `libxs_gemm_call` would succeed, zero otherwise.

```
int libxs_gemm_call(
  const libxs_gemm_config_t* config,
  const void* a, const void* b, void* c);
```

Call the GEMM kernel previously dispatched into config. Follows the documented priority (JIT > XGEMM > BLAS fallback). Returns `EXIT_SUCCESS` if a kernel was called, `EXIT_FAILURE` otherwise.

```
void libxs_gemm_release(libxs_gemm_config_t* config);
```

Release resources acquired by `libxs_gemm_dispatch` (e.g., MKL jitter). Safe to call even if dispatch was not used or returned zero.

```
void libxs_gemm_release_registry(libxs_registry_t* registry);
```

Release all GEMM configs cached in a registry (e.g., MKL jitter), then destroy the registry itself. Safe to call with NULL.

```
void libxs_gemm_batch(
  const void* a_array[], const void* b_array[], void* c_array[],
  int batchsize, const libxs_gemm_config_t* config);
```

Process a batch of GEMMs given arrays of pointers to matrices. Shape, alpha, beta, datatype, and transpose info come from `config->shape`.

```
void libxs_gemm_batch_task(
  const void* a_array[], const void* b_array[], void* c_array[],
  int batchsize, const libxs_gemm_config_t* config,
  int tid, int ntasks);
```

Per-thread form of `libxs_gemm_batch`.

```
void libxs_gemm_index(
  const void* a, const int stride_a[],
  const void* b, const int stride_b[],
        void* c, const int stride_c[],
  int index_stride, int index_base,
  int batchsize, const libxs_gemm_config_t* config);
```

Process a batch of GEMMs given index arrays into contiguous buffers. Each `stride_a[i]`, `stride_b[i]`, `stride_c[i]` is an element-offset from the respective base pointer. `index_base` selects the indexing convention: 0 for zero-based (C), 1 for one-based (Fortran). `index_stride` is the Byte-stride used to walk the stride arrays (e.g., `sizeof(int)` for packed int arrays). `index_stride=0` selects constant-stride mode: each stride array points to a single element, and batch `i` uses offset `stride[0]*i` (equivalent to the strided batch pattern). Shape, alpha, beta, datatype, and transpose info come from `config->shape`.

```
void libxs_gemm_index_task(
  const void* a, const int stride_a[],
  const void* b, const int stride_b[],
        void* c, const int stride_c[],
  int index_stride, int index_base,
  int batchsize, const libxs_gemm_config_t* config,
  int tid, int ntasks);
```

Per-thread form of `libxs_gemm_index`.

Hashing and Checksums

Header: `libxs_hash.h`

Hash functions for buffers, fixed-width keys, and strings. The default hash uses hardware-accelerated CRC32-C (SSE4.2) with a software fallback.

Buffer Hashing

```
unsigned int libxs_hash(const void* data, unsigned int size,
  unsigned int seed);
```

Produce a 32-bit hash from a byte buffer of the given size. NULL buffer is accepted (returns seed). Uses CRC32-C when SSE4.2 is available, otherwise a software table-based CRC32.

Fixed-Width Hashing

```
unsigned int libxs_hash8(unsigned int data);
unsigned int libxs_hash16(unsigned int data);
unsigned int libxs_hash32(unsigned long long data);
```

Hash an 8-, 16-, or 32-bit value. The input is split into a data part and an implicit seed derived from the remaining bits. The result is masked to the corresponding width (8, 16, or 32 bits).

String Hashing

```
unsigned long long libxs_hash_string(const char string[]);
```

64-bit hash of a character string. Short strings (up to 8 bytes) are stored directly; longer strings use two CRC32 passes combined into a 64-bit result. NULL-string is accepted (returns zero).

CRC32 (ISO 3309)

```
unsigned int libxs_hash_iso3309(const void* data,
  unsigned int size, unsigned int seed);
```

CRC-32 using the ISO 3309 polynomial (used by PNG, gzip, and similar formats). Software-only, no hardware acceleration. Pre-condition and post-XOR with 0xFFFFFFFF are the caller's responsibility. NULL buffer is accepted.

Adler-32

```
unsigned int libxs_adler32(const void* data,
  unsigned int size, unsigned int seed);
```

Adler-32 checksum (the variant used by zlib). The standard initial seed is 1. NULL buffer is accepted.

Histogram

Header: `libxs_hist.h`

Thread-safe histogram that accumulates values and provides running statistics. Values are bucketed by the first element of each inserted tuple; additional per-entry values track user-defined statistics (e.g., timing data).

Types

```
typedef struct libxs_hist_t libxs_hist_t;  /* opaque */
```

```
typedef void (*libxs_hist_update_t)(double* dst, const double* src);
```

Per-value accumulation callback. Two built-in update functions are provided: `libxs_hist_update_avg` (sliding average) and `libxs_hist_update_add` (additive accumulation).

Functions

```
libxs_hist_t* libxs_hist_create(int nbuckets, int nvals,
  const libxs_hist_update_t update[]);
```

Create a histogram with `nbuckets` resolution. `nvals` is the number of values per entry (the first value determines the bucket). The `update` array (length `nvals`) specifies the accumulation function for each value slot; NULL entries default to `libxs_hist_update_avg`. Returns NULL on failure.

```
void libxs_hist_destroy(libxs_hist_t* hist);
```

Destroy a histogram. Accepts NULL.

```
void libxs_hist_push(libxs_lock_t* lock,
  libxs_hist_t* hist, const double vals[]);
```

Insert one sample. `vals[0]` selects the bucket; remaining entries are accumulated according to the update functions. The `lock` parameter can be NULL (internal locking is used).

```
void libxs_hist_get(libxs_lock_t* lock,
  const libxs_hist_t* hist,
  const int** buckets, int* nbuckets,
  double range[2], const double** vals, int* nvals);
```

Query statistics. Output pointers can be NULL to skip fields.

```
void libxs_hist_get_percentile(libxs_lock_t* lock,
  const libxs_hist_t* hist, double percentile,
  double vals[]);
```

Query interpolated values at the given percentile. `percentile` is in the range `[0..1]` (clamped). `vals[]` is filled with interpolated results: `vals[0]` is the interpolated primary value (position along the range), and subsequent entries are linearly interpolated between adjacent buckets. Commits queued items if pending.

```
void libxs_hist_get_median(libxs_lock_t* lock,
  const libxs_hist_t* hist, double vals[]);
```

Convenience wrapper equivalent to `libxs_hist_get_percentile(lock, hist, 0.5, vals)`.

```
void libxs_hist_print(FILE* ostream,
  const libxs_hist_t* hist, const int prec[],
  const char fmt[], ...);
```

Render the histogram to a stream. `prec` controls decimal precision per value (can be NULL); a negative `prec[k]` omits that value from output, and a negative `prec[0]` skips the bucket line entirely. `fmt` is a printf-style format string for the title line, followed by optional variadic arguments. NULL `ostream` or `fmt` is accepted.

```
void libxs_hist_update_avg(double* dst, const double* src);
void libxs_hist_update_add(double* dst, const double* src);
```

Built-in update functions: `libxs_hist_update_avg` computes a sliding average (arithmetic mean), `libxs_hist_update_add` sums values.

Memory Allocation

Header: `libxs_malloc.h`

Pool-based memory allocator designed for steady-state performance: after an initial warm-up phase, allocations are served from a recycled pool without system calls.

Types

```
typedef struct libxs_malloc_info_t {
  size_t size;  /* allocated size in bytes */
} libxs_malloc_info_t;

typedef struct libxs_malloc_pool_info_t {
  size_t used;      /* memory currently in use (sum of requested sizes) */
  size_t size;      /* total allocated memory (sum of actual chunk sizes) */
  size_t peak;      /* peak memory consumption */
  size_t nactive;   /* pending (not yet freed) allocations */
  size_t nmallocs;  /* total allocation count */
} libxs_malloc_pool_info_t;

typedef struct libxs_malloc_pool_t libxs_malloc_pool_t; /* opaque */

typedef void* (*libxs_malloc_fn)(size_t size);
typedef void  (*libxs_free_fn)(void* pointer);

typedef void* (*libxs_malloc_xfn)(size_t size, const void* extra);
typedef void  (*libxs_free_xfn)(void* pointer, const void* extra);
```

Pool Management

```
libxs_malloc_pool_t* libxs_malloc_pool(libxs_malloc_fn malloc_fn, libxs_free_fn free_fn);
```

Create a memory pool. If `malloc_fn` and `free_fn` are both NULL, the standard `malloc`/`free` are used. Both must be NULL or both must be non-NULL.

```
libxs_malloc_pool_t* libxs_malloc_xpool(libxs_malloc_xfn malloc_fn, libxs_free_xfn free_fn,
  int max_nthreads);
```

Create a memory pool with extended allocator functions that receive a per-thread extra argument (see `libxs_malloc_arg`). The `max_nthreads` parameter determines the size of the internal per-thread argument table (indexed by `libxs_tid`). Both function pointers must be non-NULL and `max_nthreads` must be positive; otherwise NULL is returned.

```
void libxs_malloc_arg(libxs_malloc_pool_t* pool, const void* extra);
```

Set the per-thread extra argument for an extended pool (created by `libxs_malloc_xpool`). The pointer is stored at the calling thread's slot (`libxs_tid() % max_nthreads`) and passed to the registered `malloc_xfn`/`free_xfn` on subsequent allocations and frees from this thread. No-op for standard pools. Access is lock-free since each thread writes only its own slot.

```
void libxs_free_pool(libxs_malloc_pool_t* pool);
```

Destroy the pool and release all associated memory. Accepts NULL.

```
int libxs_malloc_pool_info(const libxs_malloc_pool_t* pool, libxs_malloc_pool_info_t* info);
```

Query aggregate pool statistics.

General Allocation

```
void* libxs_malloc(libxs_malloc_pool_t* pool, size_t size, int alignment);
```

Allocate `size` bytes from the given `pool`. `LIBXS_MALLOC_AUTO` uses automatic alignment with inline metadata. `LIBXS_MALLOC_NATIVE` preserves the allocator's native pointer (out-of-band metadata via registry). Values greater than 1 are interpreted as explicit alignment in Bytes (inline metadata). Returns NULL on failure or if `pool` is NULL.

```
void libxs_free(void* pointer);
```

Return memory to its originating pool (derived internally). Accepts NULL.

```
int libxs_malloc_info(const void* pointer, libxs_malloc_info_t* info);
```

Query the size of an allocation made with `libxs_malloc`. The pool is derived internally.

Fixed-Size Pool

A lightweight fixed-size pool for scenarios where the element size is known at initialization time.

```
void libxs_pmalloc_init(size_t size, size_t* num,
  void* pool[], void* storage);
```

Partition `storage` into `*num` chunks of `size` bytes and register them in `pool`.

```
void* libxs_pmalloc(void* pool[], size_t* num);
void* libxs_pmalloc_lock(void* pool[], size_t* num,
  libxs_lock_t* lock);
```

Pop one chunk from the pool. The `_lock` variant uses a caller-provided lock; the plain variant uses an internal lock.

```
void libxs_pfree(void* pointer, void* pool[], size_t* num);
void libxs_pfree_lock(void* pointer, void* pool[],
  size_t* num, libxs_lock_t* lock);
```

Push a chunk back into the pool.

Math Utilities

Header: `libxs_math.h`

Matrix Difference

```
typedef struct libxs_matdiff_t {
  double norm1_abs, norm1_rel;  /* one-norm */
  double normi_abs, normi_rel;  /* infinity-norm */
  double normf_rel;             /* Frobenius-norm (relative) */
  double linf_abs, linf_rel;    /* max difference (abs/rel at same element) */
  double l2_abs, l2_rel, rsq;   /* L2-norm and R-squared */
  double l1_ref, min_ref, max_ref, avg_ref, var_ref;  /* reference stats */
  double l1_tst, min_tst, max_tst, avg_tst, var_tst;  /* test stats */
  double diag_min_ref, diag_max_ref;   /* diagonal min/max (reference) */
  double diag_min_tst, diag_max_tst;   /* diagonal min/max (test) */
  double v_ref, v_tst;          /* values at max-diff location */
  int m, n, i, r;               /* location and reduction count */
} libxs_matdiff_t;
```

The fields linf_abs, linf_rel, and v_ref/v_tst always refer to the same element. For complex types, element values and statistics use the modulus; differences use the complex absolute error.

```
int libxs_matdiff(libxs_matdiff_t* info,
  libxs_data_t datatype, int m, int n,
  const void* ref, const void* tst,
  const int* ldref, const int* ldtst);
```

Compute scalar differences between two matrices. Supports all real and integer `libxs_data_t` types, plus `LIBXS_DATATYPE_C64` and `LIBXS_DATATYPE_C32` (interleaved complex; dimensions refer to complex elements).

```
double libxs_matdiff_epsilon(const libxs_matdiff_t* input);
```

Combined error margin from absolute and relative norms. Optionally logs to file via `LIBXS_MATDIFF` env var.

```
int libxs_matdiff_combine(libxs_matdiff_t* output,
  const libxs_matdiff_t* input);
```

Combine two single-matrix infos (`ref=NULL`) into a meta-diff. Per-side statistics are exact; `linf_abs` is the mean shift, `l2_abs` a statistical bound.

```
void libxs_matdiff_reduce(libxs_matdiff_t* output,
  const libxs_matdiff_t* input);
void libxs_matdiff_clear(libxs_matdiff_t* info);
```

Worst-case reduction of matdiff results. Initialize with `libxs_matdiff_clear`.

```
double libxs_matdiff_posdef(const libxs_matdiff_t* info);  /* inline */
```

Returns the smallest test-side diagonal element (positive = necessary condition for positive definiteness met). Zero if no diagonal data.

Multiset Distance

Order-independent distance between two vectors treated as multisets. The metric counts how many elements have no counterpart (within tolerance) in the other vector. The result is a non-negative integer that satisfies the metric properties: symmetry, non-negativity, and identity of indiscernibles.

For each element in b, the algorithm checks whether any element in a matches (and vice versa). The maximum of both one-sided match counts determines the distance. Elements are compared by absolute difference for real types and by complex modulus for C64/C32. The tolerance threshold is inclusive (less-than-or-equal).

```
int libxs_setdiff(libxs_data_t datatype,
  const void* a, int na,
  const void* b, int nb, double tol);
```

Supports all libxs_data_t types. Returns the number of unmatched elements, or -1 for unsupported types. The distance is at least abs(na - nb) when the vector lengths differ.

```
int libxs_setdiff_min(libxs_data_t datatype,
  const void* a, int na,
  const void* b, int nb, double* tol);
```

Minimizes the multiset distance over all tolerances using Golden Section Search (libxs_gss_min). Returns the minimum unmatched count. The pointer tol (may be NULL) receives the smallest tolerance that achieves this minimum. Supported types: F64, F32, C64, C32 only (integer types are not meaningful here; returns max(na, nb) with tol set to zero).

The distance as a function of tolerance is monotonically non-increasing (a step function with discrete drops), which makes it unimodal -- the prerequisite for Golden Section Search.

Foeppl Polynomial Fingerprint

Structural fingerprint for n-dimensional data based on Foeppl (Macaulay) bracket polynomials. For 1-D data, the fingerprint records per-derivative-order norms of the forward finite differences, normalized to the unit interval [0,1]. Higher dimensions are handled hierarchically: the innermost dimension is fingerprinted first, each child fingerprint is collapsed to a Sobolev self-norm scalar, and those scalars form the 1-D input for the next outer dimension.

Because the fingerprint is normalized to the unit interval, datasets of different lengths (or shapes) produce directly comparable fingerprints. The derivative-order decomposition captures structural features at multiple scales: order 0 measures value magnitude, order 1 measures slope/trend, order 2 measures curvature, and so on.

```
#define LIBXS_FPRINT_MAXORDER 8

typedef struct libxs_fprint_t {
  double l2[LIBXS_FPRINT_MAXORDER + 1];
  double l1[LIBXS_FPRINT_MAXORDER + 1];
  double linf[LIBXS_FPRINT_MAXORDER + 1];
  int order, n;
} libxs_fprint_t;
```

Three norm families are computed per derivative order k = 0..order:

Field	Norm	Description
`l2[k]`	L2	RMS of the k-th finite difference
`l1[k]`	L1	Mean absolute value (total variation)
`linf[k]`	Linf	Maximum absolute value (worst-case decay)

All three are normalized to the unit interval (h = 1/(n-1)). The `order` field records how many derivative orders were used; `n` is the extent of the fingerprinted dimension.

To recover raw (unnormalized) finite-difference magnitudes, use the inline helper:

```
double libxs_fprint_raw(const libxs_fprint_t* info,
  int k, double value);
```

This divides by (n-1)^k, undoing the unit-interval scaling. For k == 0, the value is returned unchanged.

The L2 norms serve comparison via the Sobolev distance. The Linf norms serve as a decay diagnostic: decaying linf[k] indicates structurally smooth data (compressible under Newton truncation), while growing linf[k] indicates unstructured data (no exploitable smoothness). The L1 norms provide a noise-robust middle ground between L2 and Linf.

```
int libxs_fprint(libxs_fprint_t* info,
  libxs_data_t datatype, const void* data,
  int ndims, const size_t shape[], const size_t stride[],
  int order, int axis);
```

Build a fingerprint from data described by shape and stride arrays. For 1-D data, pass ndims=1, shape={n}, stride=NULL. For higher dimensions, shape[0] is the innermost extent and shape[ndims-1] the outermost. When stride is NULL, contiguous storage is assumed (stride[0]=1, stride[k]=product of shape[0..k-1]). The requested order is clamped to min(order, extent-1, LIBXS_FPRINT_MAXORDER).

The axis parameter controls how multi-dimensional data is handled:

axis < 0 Hierarchical mode (default). Sweeps the innermost dimension first and collapses each level into the next outer dimension via Sobolev self-norms.

axis >= 0 Per-axis mode. Differentiates along dimension 'axis' only and takes the per-order maximum of each norm (linf, l2, l1) across all positions in the remaining dimensions.

For 1-D data, axis is ignored (both modes are equivalent).

Supported types: F64, F32, and all integer libxs_data_t types (I64, I32, U32, I16, U16, I8, U8 -- promoted to double internally).

```
double libxs_fprint_diff(
  const libxs_fprint_t* a, const libxs_fprint_t* b,
  const double* weights);
```

Weighted Sobolev distance between two fingerprints: d = sqrt(sum over k of weights[k] * (a->l2[k] - b->l2[k])^2). The number of orders compared is min(a->order, b->order) + 1. If weights is NULL, default weights w(k) = 1/k! are used, which naturally dampens higher-order (noisier) derivatives.

The distance is a metric: symmetric, non-negative, zero if and only if the fingerprints are identical. The same function serves both as a distance measure and as a fingerprint comparator since the fingerprint is the decomposed form of the Sobolev norm.

Golden Section Search

```
double libxs_gss_min(
  double (*fn)(double x, const void* data),
  const void* data,
  double x0, double x1, double* xmin, int maxiter);
```

Minimizes a unimodal function fn on the interval [x0, x1]. The callback receives x and an opaque context pointer. Returns f(x*) where x* is the minimizer; xmin (may be NULL) receives x*.

The bracket shrinks by factor phi = (sqrt(5)-1)/2 per iteration, reusing one evaluation from the previous step. Convergence is reached when the bracket collapses to machine precision or maxiter iterations are exhausted.

Number Theory

```
size_t libxs_gcd(size_t a, size_t b);
size_t libxs_lcm(size_t a, size_t b);
```

GCD (returns 1 for gcd(0,0)) and LCM.

```
int libxs_primes_u32(unsigned int num,
  unsigned int num_factors_n32[], int num_factors_max);
```

Prime factors of `num` in ascending order. Returns factor count.

```
size_t libxs_coprime(size_t n, size_t minco);
size_t libxs_coprime_bias(size_t n, double bias);
size_t libxs_coprime2(size_t n); /* inline: coprime_bias(n, 0) */
```

`libxs_coprime`: co-prime of `n` not exceeding `minco`.

`libxs_coprime_bias`: co-prime of `n` selected by bias in [-1, +1] via a logarithmic mapping (target = n^((1+bias)/2)). bias=-1 selects the smallest non-trivial coprime (maximum displacement), bias=0 selects near sqrt(n) (balanced, same as libxs_coprime2), bias=+1 selects near n/2 (near-alternation). Monotonic: larger bias always yields a larger (or equal) coprime.

`libxs_coprime2`: inline convenience for `libxs_coprime_bias(n, 0)`.

```
unsigned int libxs_remainder(unsigned int a, unsigned int b,
  const unsigned int* limit, const unsigned int* remainder);
```

Smallest multiple of `b` minimizing remainder mod `a`.

```
unsigned int libxs_product_limit(unsigned int product,
  unsigned int limit, int is_lower);
```

Sub-product of prime factors within `limit` (0/1-Knapsack).

Scalar Utilities

```
double libxs_kahan_sum(double value, double* accumulator,
  double* compensation);
```

Kahan compensated summation.

```
unsigned int libxs_isqrt_u64(unsigned long long x);
unsigned int libxs_isqrt_u32(unsigned int x);
unsigned int libxs_isqrt2_u32(unsigned int x);
```

Integer square root. The isqrt2 variant returns a factor of x.

```
double libxs_pow2(int n);
```

2ˆn via IEEE-754 exponent. Valid for n in [-1022, 1023]. Returns 0.0 for underflow (subnormals flushed to zero) and +Inf for overflow.

Modular Arithmetic

```
unsigned int libxs_mod_inverse_u32(unsigned int a, unsigned int m);
```

Modular inverse (extended Euclidean). Requires gcd(a, m) = 1.

```
unsigned int libxs_barrett_rcp(unsigned int p);
unsigned int libxs_barrett_pow18(unsigned int p);
unsigned int libxs_barrett_pow36(unsigned int p);
```

Barrett reduction constants for modulus `p`.

```
unsigned int libxs_mod_u32(uint32_t x, unsigned int p,
  unsigned int rcp);                          /* inline */
unsigned int libxs_mod_u64(uint64_t x, unsigned int p,
  unsigned int rcp, unsigned int pow18,
  unsigned int pow36);                        /* inline */
```

Fast modular reduction via Barrett. The 64-bit variant uses a radix-2ˆ18 split.

BF16 Conversion

```
typedef uint16_t libxs_bf16_t;
libxs_bf16_t libxs_round_bf16(double x);   /* inline */
double libxs_bf16_to_f64(libxs_bf16_t v);  /* inline */
```

Round to BF16 (round-to-nearest-even) and expand to double. Uses native `__bf16` when available (`LIBXS_BF16`), otherwise portable bit manipulation.

Memory Operations

Header: `libxs_mem.h`

Byte-level memory macros, pointer alignment queries, buffer comparison, and matrix copy/transposition.

Memory Macros

```
LIBXS_MEMSET(DST, SRC, SIZE)
```

Set SIZE bytes at DST to the value SRC, unrolled at compile-time.

```
LIBXS_MEMZERO(DST)
```

Zero all bytes of *DST (size derived from sizeof(*DST)).

```
LIBXS_MEMCPY(DST, SRC, SIZE)
```

Copy SIZE bytes from SRC to DST, unrolled at compile time.

```
LIBXS_ASSIGN(DST, SRC)
```

Copy sizeof(*SRC) bytes from SRC to DST. Asserts sizeof(*SRC) <= sizeof(*DST).

```
LIBXS_MEMSWP(PTR_A, PTR_B, SIZE)
```

Swap SIZE bytes between PTR_A and PTR_B. Asserts PTR_A != PTR_B.

```
LIBXS_VALUE_ASSIGN(DST, SRC)
LIBXS_VALUE_SWAP(A, B)
```

Operate on L-values directly (address-of is taken internally). VALUE_ASSIGN can cast away const qualifiers. VALUE_SWAP asserts equal size.

Offset and Alignment

```
size_t libxs_offset(size_t ndims, const size_t offset[],
  const size_t shape[], size_t* size);
```

Compute the linear offset of an n-dimensional coordinate within a shape. Optionally returns the total linear size of the shape.

```
int libxs_aligned(const void* ptr, const size_t* inc,
  int* alignment);
```

Check whether ptr is SIMD-aligned (optionally considering the next access at ptr + *inc). Optionally writes the pointer alignment in bytes to *alignment.

Comparison

```
unsigned char libxs_diff(const void* a, const void* b,
  unsigned char size);
```

Compare two short buffers. Returns zero when equal, non-zero otherwise. Uses SSE/AVX2/AVX-512 when available.

```
unsigned int libxs_diff_n(const void* a, const void* bn,
  unsigned char elemsize, unsigned char stride,
  unsigned int hint, unsigned int count);
```

Compare a against count elements in bn. Returns the first index that matches a, or count if none match. hint supplies the initial search position.

```
int libxs_memcmp(const void* a, const void* b, size_t size);
```

Boolean variant of the C memcmp. Returns zero when equal. Uses SSE/AVX2/AVX-512 when available.

Matrix Copy and Transposition

```
void libxs_matcopy(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo);
void libxs_matcopy_task(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

Copy an m-by-n matrix from in to out with leading dimensions ldi and ldo. If in is NULL the destination is zeroed. The _task variant distributes work across ntasks threads (caller passes its tid).

```
void libxs_otrans(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo);
void libxs_otrans_task(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

Out-of-place matrix transposition. The _task variant distributes work across ntasks threads.

```
void libxs_itrans(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo, void* scratch);
void libxs_itrans_task(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo, void* scratch,
  int tid, int ntasks);
```

In-place matrix transposition (square or via scratch buffer). scratch can be NULL (auto-allocated).

```
void libxs_itrans_batch(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo,
  int index_base, int index_stride,
  const int stride[], int batchsize,
  int tid, int ntasks);
```

Batch of in-place matrix transpositions (per-thread form).

MetaImage I/O

Header: `libxs_mhd.h`

Read and write image data in the MetaImage (MHD/MHA) format. Files are compatible with ITK, ITK-SNAP, 3D Slicer, ParaView, and similar tools.

Types

```
typedef enum libxs_mhd_element_conversion_hint_t {
  LIBXS_MHD_ELEMENT_CONVERSION_DEFAULT,
  LIBXS_MHD_ELEMENT_CONVERSION_MODULUS
} libxs_mhd_element_conversion_hint_t;
```

Hint controlling element conversion behaviour (default clamping vs. modulus mapping).

```
typedef struct libxs_mhd_element_handler_info_t {
  libxs_data_t type;
  libxs_mhd_element_conversion_hint_t hint;
} libxs_mhd_element_handler_info_t;
```

Destination type and conversion hint, passed to built-in or custom element handlers.

```
typedef int (*libxs_mhd_element_handler_t)(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max);
```

Per-element callback for reading or converting image data. The value range (`src_min`, `src_max`) is available for scaling.

```
typedef struct libxs_mhd_info_t {
  size_t ndims;         /* dimensionality */
  size_t ncomponents;   /* interleaved image channels */
  libxs_data_t type;    /* pixel element type */
  size_t header_size;   /* header size in bytes (LOCAL data) */
} libxs_mhd_info_t;
```

Image descriptor populated by `libxs_mhd_read_header` and consumed by read/write.

```
typedef struct libxs_mhd_write_info_t {
  const libxs_mhd_element_handler_info_t* handler_info;
  libxs_mhd_element_handler_t handler;
  const char* extension_header;
  const void* extension;
  size_t extension_size;
} libxs_mhd_write_info_t;
```

Optional write parameters (conversion, custom handler, extension data). NULL is accepted for all-defaults.

Element Helpers

```
int libxs_mhd_element_conversion(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max);
```

Built-in element conversion. Scales values when `src_min` and `src_max` are non-NULL; otherwise clamps to the destination type.

```
int libxs_mhd_element_comparison(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max);
```

Check an in-memory buffer against file content (element-wise comparison with optional type conversion).

Type Queries

```
const char* libxs_mhd_typename(libxs_data_t type,
  const char** ctypename);
```

Return the MHD element-type name (e.g. `"MET_FLOAT"`) for the given `libxs_data_t`. Optionally writes the C type name. Returns NULL for unknown types.

```
libxs_data_t libxs_mhd_typeinfo(const char elemname[]);
```

Return the `libxs_data_t` for a given MHD element-type string.

Read

```
int libxs_mhd_read_header(const char header_filename[],
  size_t filename_max_length, char filename[],
  libxs_mhd_info_t* info, size_t size[],
  char extension[], size_t* extension_size);
```

Parse an MHD header file. `info->ndims` is an in/out parameter (maximum on input, actual on output). `filename` receives the data-file path (may differ from the header when data is stored separately). Extension data is optional.

```
int libxs_mhd_read(const char filename[],
  const size_t offset[], const size_t size[],
  const size_t pitch[], const libxs_mhd_info_t* info,
  void* data,
  const libxs_mhd_element_handler_info_t* handler_info,
  libxs_mhd_element_handler_t handler);
```

Read image data into `data`. Optional `handler_info` triggers type conversion; optional `handler` supplies a custom per-element callback (can also serve as a comparison function without allocating a buffer).

Write

```
int libxs_mhd_write(const char filename[],
  const size_t offset[], const size_t size[],
  const size_t pitch[], libxs_mhd_info_t* info,
  const void* data,
  const libxs_mhd_write_info_t* write_info);
```

Write image data in the extended MHD format. `info->header_size` is updated on output. Pass NULL for `write_info` when no conversion or extension is needed.

When the environment variable `LIBXS_MHD_PNG` is set and the image is two-dimensional, an uncompressed PNG file is written alongside the MHD output (same base name, `.png` extension). Supported channel counts are 1 (grayscale), 3 (RGB), and 4 (RGBA) at 8-bit depth. Source data of any element type is converted to unsigned 8-bit using the built-in element conversion with automatic min/max scaling.

Visit LIBXSTREAM

Start Presentation

Permutation and Sorting

Header: `libxs_perm.h`

Permutation-based data reordering: deterministic co-prime shuffling (in-place and out-of-place with optional SIMD gather) and smoothness-optimized row permutations for matrices.

Shuffling

All three functions use a co-prime stride to produce a fixed, deterministic permutation of count elements. The stride must be co-prime to count; passing NULL selects libxs_coprime2(count).

```
int libxs_shuffle(void* inout, size_t elemsize, size_t count,
  const size_t* shuffle, const size_t* nrepeat);
```

In-place shuffle of count elements of elemsize bytes each. nrepeat controls the number of successive permutation applications (NULL or pointing to 1 means one pass). Returns EXIT_SUCCESS or EXIT_FAILURE.

```
int libxs_shuffle2(void* dst, const void* src, size_t elemsize,
  size_t count, const size_t* shuffle, const size_t* nrepeat);
```

Out-of-place shuffle from src to dst. dst and src must not overlap. If *nrepeat is zero an ordinary copy is performed. Uses AVX2/AVX-512 gather instructions when available for 4- and 8-byte elements.

```
size_t libxs_unshuffle(size_t count, const size_t* shuffle);
```

Return the number of libxs_shuffle2 applications needed to restore the original element order for the given count and co-prime stride.

Smooth Sorting

```
#define LIBXS_SORT_NONE     0
#define LIBXS_SORT_IDENTITY 1
#define LIBXS_SORT_NORM     2
#define LIBXS_SORT_MEAN     3
#define LIBXS_SORT_GREEDY   4
```

Method constants for libxs_sort_smooth.

```
int libxs_sort_smooth(int method, int m, int n,
  const void* mat, int ld, libxs_data_t datatype, int* perm);
```

Compute a row permutation that reorders the rows of an m-by-n column-major matrix for smoothness (decaying forward differences between consecutive rows).

Parameters:

method Sorting strategy (see constants above). m, n Matrix dimensions (m rows, n columns). mat Column-major matrix data (read-only). ld Leading dimension (ld >= m). datatype Element type (F64, F32, I32, U32, I16, U16, I8, U8). perm Output array of m ints; perm[i] = source row for position i after reordering.

Methods:

NONE No permutation (early return). IDENTITY Identity permutation (perm[i] = i). NORM Sort rows by ascending L1-norm. MEAN Sort rows by ascending column mean. GREEDY Nearest-neighbor chain: starting from row 0, each step picks the unvisited row with the smallest Euclidean distance to the current row.

Returns EXIT_SUCCESS or EXIT_FAILURE.

Registry

Header: `libxs_reg.h`

Thread-safe key-value store backed by a hash table with per-thread caching.

Configuration Macros

| Macro | Default | Description | |:|:|:| | `LIBXS_REGKEY_MAXSIZE` | 64 | Maximum key size in bytes | | `LIBXS_REGISTRY_NBUCKETS` | 64 | Initial number of hash-table buckets (must be a power of two) | | `LIBXS_REGCACHE_NENTRIES` | 16 | Thread-local cache entries per thread (power of two; 0 disables caching) |

Types

```
typedef struct libxs_registry_t libxs_registry_t;   /* opaque */

typedef struct libxs_registry_info_t {
  size_t capacity, size, nbytes;
} libxs_registry_info_t;
```

Lifetime

```
libxs_registry_t* libxs_registry_create(void);
```

Create a new registry. Returns `NULL` in case of an error.

```
void libxs_registry_destroy(libxs_registry_t* registry);
```

Destroy a registry and release all entries.

```
libxs_lock_t* libxs_registry_lock(libxs_registry_t* registry);
```

Return a pointer to the registry's internal lock. The returned lock can be passed as the `lock` argument to other registry functions (e.g., to hold the lock across multiple operations).

Iteration

```
void* libxs_registry_begin(libxs_registry_t* registry,
  const void** key, size_t* cursor);
void* libxs_registry_next(libxs_registry_t* registry,
  const void** key, size_t* cursor);
```

Enumerate entries. Initialize `*cursor` to 0 before the first `begin` call. Each call returns the value pointer and writes the key pointer to `*key`, or returns NULL when iteration is complete.

Access

```
void* libxs_registry_set(libxs_registry_t* registry,
  const void* key, size_t key_size,
  const void* value_init, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Insert or update a key-value pair. The key must be binary-reproducible (zero-initialize padding). `value_init` may be NULL to defer initialization. Re-registering an existing key reallocates if the new value is larger. When `lock` is NULL the registry's internal lock is used. Returns a pointer to the stored value.

```
void* libxs_registry_get(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Look up a value by key. Returns NULL if not found.

```
int libxs_registry_get_copy(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  void* value_out, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Thread-safe query: copies up to `value_size` bytes of the stored value into `value_out` under the lock. Unlike `libxs_registry_get`, the caller never receives a raw pointer into the registry's internal storage, making this safe under concurrent modifications. Returns non-zero if the key was found, zero otherwise.

```
int libxs_registry_has(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Check whether a key exists. Non-zero if found.

```
size_t libxs_registry_value_size(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Query the stored value size in bytes. Returns 0 if not found.

```
void libxs_registry_remove(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Remove a key-value pair and release its memory.

```
int libxs_registry_extract(libxs_registry_t* registry,
  const void* key, size_t key_size,
  void* value_out, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Atomically retrieve and remove a key-value pair. Copies up to `value_size` bytes of the stored value into `value_out` (may be NULL to discard the value), then removes the entry and releases its memory. The lookup, copy, and removal are performed under a single lock hold, eliminating the race that exists when calling `libxs_registry_get` followed by `libxs_registry_remove` separately. Returns non-zero if the key was found, zero otherwise.

Status

```
int libxs_registry_info(libxs_registry_t* registry,
  libxs_registry_info_t* info);
```

Query registry capacity, number of entries, and total bytes stored.

Random Number Generator

Header: `libxs_rng.h`

Thread-safe pseudo-random number generation (SplitMix64 with per-thread TLS state).

Matrix Initialization Macros

```
LIBXS_MATRNG(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
LIBXS_MATRNG_SEQ(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
LIBXS_MATRNG_OMP(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
```

Fill a column-major matrix (NROWS x NCOLS, leading dimension LD) with deterministic values. The `_OMP` variant parallelizes with OpenMP; `_SEQ` is serial.

ESPAN (exponent span) selects the initialization mode:

ESPAN == 0 Shuffle mode. A coprime permutation maps each linear index to a unique value in [-|SCALE|, +|SCALE|]. Covers the full LD*NCOLS range (including padding).

ESPAN != 0 Adversarial exponent-span mode for stress-testing floating-point emulation (Ozaki scheme, etc.). Base values are shuffled in [1, 2), then each column j is scaled by

```
  2^(sign(ESPAN) \* floor(|ESPAN| \* j / (NCOLS-1)))

so column 0 has exponent 0 and the last column has
exponent +/-|ESPAN|. Padding rows \[NROWS, LD) are
zero-filled.

Use +ESPAN for matrix A and -ESPAN for matrix B so
that A\*B remains well-conditioned while each operand
individually has adversarial exponent range.
```

Example -- adversarial DGEMM test with exponent span 512:

```
int espan = 512;
LIBXS_MATRNG(int, double, +espan, a, m, k, lda, 1.0);
LIBXS_MATRNG(int, double, -espan, b, k, n, ldb, 1.0);
```

Functions

```
void libxs_rng_set_seed(unsigned int seed);
```

Set the seed of the calling thread's PRNG state. Each thread maintains independent state via TLS. Unseeded threads start with a deterministic default (seed = 1).

```
unsigned int libxs_rng_u32(unsigned int n);
```

Return a pseudo-random value in [0, n) with uniform distribution (Lemire's nearly-divisionless method). Thread-safe.

```
double libxs_rng_f64(void);
```

Return a double-precision value in [0, 1) with full 53-bit mantissa resolution. Thread-safe.

```
void libxs_rng_seq(void* data, size_t nbytes);
```

Fill a buffer with pseudo-random bytes. Thread-safe.

String Utilities

Header: `libxs_str.h`

Case-insensitive string search, edit distance, word-level similarity scoring, and value formatting.

Substring Search

```
const char* libxs_stristrn(const char a[], const char b[],
  size_t maxlen);
const char* libxs_stristr(const char a[], const char b[]);
```

Case-insensitive substring search. stristrn limits the match length to maxlen characters of b. Returns a pointer to the first match in a, or NULL.

Edit Distance

```
int libxs_stridist(const char a[], const char b[]);
```

Case-insensitive character-level edit distance (Levenshtein) between two strings. Returns the minimum number of single-character insertions, deletions, or substitutions to transform a into b (ignoring case). Returns -1 for NULL input.

Word Matching

```
int libxs_strimatch(const char a[], const char b[],
  const char delims[], int* count);
```

Word-level fuzzy matching. Counts how many words in a (or b) appear in the other string (case-insensitive, symmetric). Optional delims define word separators (default: space, tab, semicolon, comma, colon, dash). Optional count receives the total word count. Returns -1 for invalid input.

Word Similarity

```
typedef enum libxs_strisimilar_t {
  LIBXS_STRISIMILAR_GREEDY,
  LIBXS_STRISIMILAR_TWOOPT,
  LIBXS_STRISIMILAR_DEFAULT = LIBXS_STRISIMILAR_GREEDY
} libxs_strisimilar_t;

int libxs_strisimilar(const char a[], const char b[],
  const char delims[], libxs_strisimilar_t kind, int* order);
```

Word-level similarity score combining edit distance and word-order analysis.

Strings are split into words using the same delimiters as libxs_strimatch. Each word in a is matched to a word in b via minimum-cost bipartite matching, where the cost of a pair is the character-level Levenshtein distance (case-insensitive). Unmatched words (when the strings have different word counts) contribute their full length.

The matching strategy is selected by kind:

GREEDY Picks the globally cheapest pair first. TWOOPT Refines the greedy result by iteratively swapping pairs whenever a swap reduces total cost.

The optional order output receives the number of pairwise inversions among matched words (Kendall tau distance), measuring how much the word order differs (0 = same order).

Returns the total edit distance (0 for identical word sets in any order), or -1 for invalid input.

Value Formatting

```
size_t libxs_format_value(char buffer[], int buffer_size,
  size_t nbytes, const char scale[], const char* unit, int base);
```

Format a scalar value with SI-style scaling. Example:

libxs_format_value(buf, sizeof(buf), nbytes, "KMGT", "B", 10)

produces a human-readable byte count such as "128 KB". Returns the value in the selected unit so the caller can decide whether to print the buffer.

Synchronization

Header: `libxs_sync.h`

Thread-local storage, atomic operations, lock abstractions (spin/mutex/rwlock/atomic), file locking, and stdio synchronization.

Thread-Local Storage

`LIBXS_TLS`

Storage-class qualifier for thread-local variables. Maps to `__thread`, `__declspec(thread)`, or `thread_local` depending on the compiler. Defined as empty when TLS is unavailable or disabled (`LIBXS_NO_TLS`).

Atomic Operations

The header provides a suite of macros for atomic loads, stores, compare-and-swap, and arithmetic. The implementation selects among GCC builtins (`__atomic_*`), legacy GCC sync builtins, Windows Interlocked intrinsics, or no-op fallbacks depending on the compiler and `LIBXS_SYNC` setting.

| Macro | Description | |:|:| | `LIBXS_ATOMIC_LOAD(SRC, KIND)` | Atomic load | | `LIBXS_ATOMIC_STORE(DST, VALUE, KIND)` | Atomic store | | `LIBXS_ATOMIC_STORE_ZERO(DST, KIND)` | Atomic store of zero | | `LIBXS_ATOMIC_CMPSWP(DST, OLDVAL, NEWVAL, KIND)` | Compare-and-swap | | `LIBXS_ATOMIC_FETCH_OR(DST, VALUE, KIND)` | Fetch-and-or | | `LIBXS_ATOMIC_FETCH_ADD(DST, VALUE, KIND)` | Fetch-and-add | | `LIBXS_ATOMIC_FETCH_SUB(DST, VALUE, KIND)` | Fetch-and-subtract | | `LIBXS_ATOMIC_ADD_FETCH(DST, VALUE, KIND)` | Add-and-fetch | | `LIBXS_ATOMIC_SUB_FETCH(DST, VALUE, KIND)` | Subtract-and-fetch | | `LIBXS_ATOMIC_TRYLOCK(DST, KIND)` | Try-lock (returns acquired state) | | `LIBXS_ATOMIC_ACQUIRE(DST, NPAUSE, KIND)` | Spin-acquire with backoff | | `LIBXS_ATOMIC_RELEASE(DST, KIND)` | Release (store zero) | | `LIBXS_ATOMIC_SYNC(KIND)` | Full memory fence |

The `KIND` parameter selects the memory order but is ignored on most backends (sequential consistency is always used).

Lock Abstraction

```
LIBXS_LOCK_TYPE(KIND)
LIBXS_LOCK_INIT(KIND, LOCK, ATTR)
LIBXS_LOCK_DESTROY(KIND, LOCK)
LIBXS_LOCK_ACQUIRE(KIND, LOCK)
LIBXS_LOCK_TRYLOCK(KIND, LOCK)
LIBXS_LOCK_RELEASE(KIND, LOCK)
```

Generic lock interface parameterized by `KIND`:

| KIND | Backend | |:|:| | `LIBXS_LOCK_SPINLOCK` | Atomic spin-lock | | `LIBXS_LOCK_MUTEX` | Pthreads / Windows CRITICAL_SECTION | | `LIBXS_LOCK_RWLOCK` | Pthreads reader-writer lock | | `LIBXS_LOCK_ATOMIC` | Lightweight atomic lock |

The default lock kind used by the library is `LIBXS_LOCK` (resolves to one of the above based on build configuration).

```
typedef LIBXS_LOCK_TYPE(LIBXS_LOCK) libxs_lock_t;
```

General-purpose lock type. Instances of `libxs_lock_t` are used by the registry and other library components; users may also create their own.

Reader-writer variants are available for `LIBXS_LOCK_RWLOCK`:

```
LIBXS_LOCK_ACQREAD(KIND, LOCK)
LIBXS_LOCK_RELREAD(KIND, LOCK)
LIBXS_LOCK_TRYREAD(KIND, LOCK)
```

File Locking

```
LIBXS_FLOCK(FILE)
LIBXS_FUNLOCK(FILE)
```

Per-file locking for thread-safe I/O. Maps to `flockfile`/`funlockfile` on POSIX, `_lock_file`/`_unlock_file` on Windows, or no-ops when synchronization is disabled.

Functions

```
unsigned int libxs_nranks(void);
```

Return the number of MPI ranks.

```
unsigned int libxs_nrank(void);
```

Return the MPI rank of the calling process.

```
unsigned int libxs_rid(void);
```

Return a rank ID of the calling process.

```
unsigned int libxs_pid(void);
```

Return the process ID of the calling process.

```
unsigned int libxs_tid(void);
```

Return a zero-based, consecutive thread ID for the calling thread. TID = 0 does not necessarily correspond to the main thread.

```
void libxs_stdio_acquire(void);
void libxs_stdio_release(void);
```

Acquire/release a global lock around console output. The macros `LIBXS_STDIO_ACQUIRE()` and `LIBXS_STDIO_RELEASE()` expand to these calls.

Timer

Header: `libxs_timer.h`

High-resolution timing via a calibrated TSC when available, with an OS real-time clock fallback.

Types

```
typedef unsigned long long libxs_timer_tick_t;
```

Integer type representing a single timer sample.

```
typedef struct libxs_timer_info_t {
  int tsc;   /* non-zero if a calibrated TSC is used */
} libxs_timer_info_t;
```

Functions

```
int libxs_timer_info(libxs_timer_info_t* info);
```

Query timer properties. `info->tsc` is non-zero when RDTSC (x86), CNTVCT_EL0 (AArch64), or MFTB (PPC64) is available. Triggers lazy library initialization if necessary.

```
libxs_timer_tick_t libxs_timer_tick(void);
```

Sample the current tick of the monotonic timer. The tick unit is platform-specific; convert with `libxs_timer_duration` or `libxs_timer_ncycles`.

```
libxs_timer_tick_t libxs_timer_ncycles(
  libxs_timer_tick_t tick0, libxs_timer_tick_t tick1);   /* inline */
```

Unsigned difference `tick1 - tick0`, handling wrap-around safely. Implemented via `LIBXS_DELTA`.

```
double libxs_timer_duration(
  libxs_timer_tick_t tick0, libxs_timer_tick_t tick1);
```

Convert a pair of ticks to elapsed wall-clock time in seconds.

Low-Level Utilities and SIMD

Header: `libxs_utils.h`

Target-attribute machinery, ISA feature gates, intrinsic fix-ups, bit-scan operations, and AVX-512 inline helpers.

Target Attributes

`LIBXS_INTRINSICS(TARGET)`

Expands to a `__attribute__((target(...)))` clause for the given ISA level. Used to compile individual functions for a higher ISA than the baseline without changing compiler flags. The macro is a no-op when the baseline already covers the requested level or when the compiler does not support multi-versioning.

`LIBXS_ATTRIBUTE_TARGET(TARGET)`

Lower-level macro that resolves `TARGET` (an integer ISA constant such as `LIBXS_X86_AVX512`) to the corresponding target string, e.g. `target("avx2,fma,avx512f,...")`.

ISA Feature Gates

Preprocessor symbols defined when the compiler can generate code for a given ISA:

Macro	ISA level
:	:
`LIBXS_INTRINSICS_X86`	x86 baseline (SSE2)
`LIBXS_INTRINSICS_SSE3`	SSE3
`LIBXS_INTRINSICS_SSE42`	SSE4.2
`LIBXS_INTRINSICS_AVX`	AVX
`LIBXS_INTRINSICS_AVX2`	AVX2 + FMA
`LIBXS_INTRINSICS_AVX512`	AVX-512 (F + CD + DQ + BW + VL + VNNI)

`LIBXS_STATIC_TARGET_ARCH` records the highest ISA provided by the compiler flags. `LIBXS_MAX_STATIC_TARGET_ARCH` records the highest ISA reachable via target attributes. Target attributes (`LIBXS_ATTRIBUTE_TARGET_*`) are defined up to `LIBXS_X86_AVX10_512` depending on compiler version.

Intrinsic Fix-ups

Portability wrappers for intrinsics whose signatures differ across compilers:

Macro	Purpose
:	:
`LIBXS_INTRINSICS_LOADU_SI128`	`_mm_loadu_si128`
`LIBXS_INTRINSICS_MM512_LOAD_PS/PD`	Unaligned 512-bit float/double load
`LIBXS_INTRINSICS_MM512_STREAM_*`	Non-temporal 512-bit stores
`LIBXS_INTRINSICS_MM256_STORE_EPI32`	256-bit integer store
`LIBXS_INTRINSICS_MM512_SET_EPI16`	Portable 512-bit set from 32 int16 values
`LIBXS_INTRINSICS_MM512_UNDEFINED_*`	Undefined-value initializers (zero in debug)
`LIBXS_INTRINSICS_MM512_EXTRACTI64X4_EPI64`	256-bit extract from 512-bit
`LIBXS_INTRINSICS_MM512_ABS_PS`	Absolute value of packed floats
`LIBXS_INTRINSICS_MM512_MASK_I32GATHER_EPI32`	Masked gather
`LIBXS_INTRINSICS_MM512_STORE/LOAD_MASK*`	Mask register I/O (8/16/32/64-bit)
`LIBXS_INTRINSICS_MM512_CVTU32_MASK*`	Convert uint32 to mask register

Bit-Scan Operations

```
LIBXS_INTRINSICS_BITSCANFWD32(N)   /* count trailing zeros, 32-bit */
LIBXS_INTRINSICS_BITSCANFWD64(N)   /* count trailing zeros, 64-bit */
LIBXS_INTRINSICS_BITSCANBWD32(N)   /* bit index of highest set bit, 32-bit */
LIBXS_INTRINSICS_BITSCANBWD64(N)   /* bit index of highest set bit, 64-bit */
```

Use GCC `__builtin_ctz`/`__builtin_clz` when available, Windows `_BitScanForward`/`_BitScanReverse` intrinsics on MSVC, or portable software fallbacks (`*_SW` variants). All return 0 when `N` is 0.

Derived Macros

```
LIBXS_NBITS(N)     /* minimum bits to represent N */
LIBXS_ISQRT2(N)    /* fast power-of-two approximation of sqrt(N) */

unsigned int LIBXS_ILOG2(unsigned long long n);   /* ceil(log2(n)) */
```

`LIBXS_ILOG2` is a function (not a macro) and handles the n = 0 case.

MXCSR (FP Environment)

Macros for preserving and restoring the x86 MXCSR register (floating-point control/status). On non-x86 platforms these are no-ops.

| Macro | Description | |:|:| | `LIBXS_MXCSR_FTZ` | Flush-to-zero flag (0x8000 on x86, 0 otherwise) | | `LIBXS_MXCSR_DAZ` | Denormals-are-zero flag (0x0040 on x86, 0 otherwise) | | `LIBXS_MXCSR_GET()` | Read the MXCSR register (`_mm_getcsr()` on x86, 0 otherwise) | | `LIBXS_MXCSR_SET(VALUE)` | Write the MXCSR register (`_mm_setcsr()` on x86, no-op otherwise) |

AVX-512 Inline Functions

Available only when `LIBXS_INTRINSICS_AVX512` is defined.

```
__m512i libxs_mulhi_epu32(__m512i a, __m512i b);   /* inline */
```

Unsigned 32-bit high-multiply for 16 lanes: floor(a * b / 2^32). Emulates the missing `_mm512_mulhi_epu32` via even/odd `_mm512_mul_epu32`.

```
__m512i libxs_mod_u32x16(__m512i x,
  unsigned int p, unsigned int rcp);               /* inline */
```

Vectorized Barrett reduction: x mod p for 16 uint32 lanes. `rcp` is the Barrett reciprocal from `libxs_barrett_rcp()`. This is the SIMD counterpart of the scalar `libxs_mod_u32`.

Appendix

Scripts

Build Support

tool_source.sh -- Generate `libxs_source.h` (header-only amalgamation). Called by `make headers`.

tool_version.sh -- Extract version from Git tags. Generates `libxs_version.h`.

tool_pkgconfig.pc.in -- Template for the installed `.pc` file.

Code Quality

tool_analyze.sh -- Run cppcheck on `src/`.

tool_checkabi.sh -- Compare exported symbols against a reference.

tool_clangformat.sh -- Run clang-format and shellcheck.

tool_normalize.sh -- Strip trailing whitespace, fix line endings.

Development

tool_cpuinfo.sh -- Print CPU topology (sockets, cores, threads).

tool_getenvars.sh -- List environment variables used in the source tree.

tool_pexec.sh -- Parallel command execution with CPU affinity (`-h` for options).

tool_gitaddforks.sh -- Add GitHub forks as Git remotes.

tool_gitprune.sh -- Aggressive Git housekeeping (gc, fsck, repack).